AWS Academy Lab Project - Cloud Web Application Builder

By

Rawa Hasan

Business scenario overview

- Provide problem or opportunity statement.

The scenario involves planning, designing, building and deploying a web application to the AWS Cloud in a manner consistent with the AWS Well-Architected Framework's best practices. During the peak admissions period, the application must support thousands of users and offer high availability, scalability, load balancing, security and performance.

Business scenario overview

- Describe solution requirements.

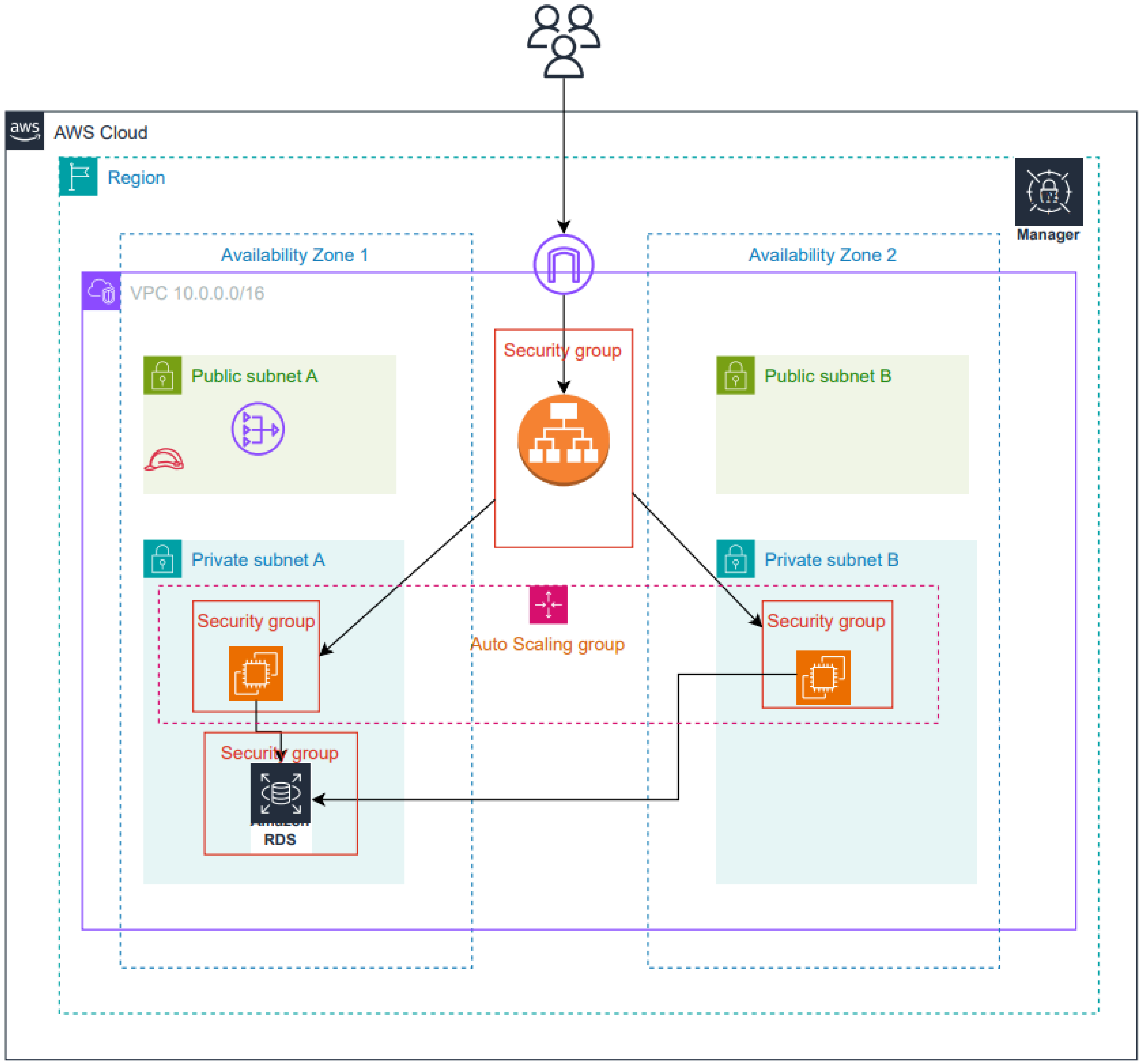
1. Create an architectural diagram depicting the various AWS services and how they interact with each other.
2. Use the AWS Pricing Calculator to estimate the cost of using the services.
3. Deploy a functional web application that runs on a single virtual machine and is supported by a relational database.
4. Design a web application with separate layers, such as the web server and database.
5. Configure a virtual network to host a publicly accessible and secure web application.
6. Deploy a web application with load distributed across multiple web servers.
7. Configure the appropriate network security settings for the web servers and database.
8. Implement high availability and scalability in the deployed solution.

Phase 1: Planning the design and estimating cost

Task 1: Creating an architectural diagram

Plan the design and implementation by creating an architecture diagram for the desired solution.

Architecture diagram of the solution



Cost estimation

Estimate summary

Upfront cost	Monthly cost	Total 12 months cost
0.00 USD	79.87 USD	958.44 USD Includes upfront cost

Detailed Estimate

Name	Group	Region	Upfront cost	Monthly cost
Amazon EC2	-	US East (N. Virginia)	0.00 USD	14.02 USD

Status -

Description: -

Config summary Tenancy (Shared Instances), Operating system (Ubuntu Pro), Workload (Consistent, Number of instances: 2), Advance EC2 instance (t3a.micro), Pricing strategy (1yr No Upfront), Enable monitoring (disabled), DT Inbound: Not selected (0 TB per month), DT Outbound: Not selected (0 TB per month), DT Intra-Region: (0 TB per month)

Name	Group	Region	Upfront cost	Monthly cost
Amazon Virtual Private Cloud (VPC)	-	US East (N. Virginia)	0.00 USD	7.30 USD

Status -

Description: -

Config summary Number of In-use public IPv4 addresses (2)

Name	Group	Region	Upfront cost	Monthly cost
Elastic Load Balancing	-	US East (N. Virginia)	0.00 USD	22.27 USD

Status -

Description: -

Config summary Number of Application Load Balancers (1)

Name	Group	Region	Upfront cost	Monthly cost
Amazon RDS for MySQL	-	US East (N. Virginia)	0.00 USD	35.88 USD

Status -

Description: -

Config summary Storage amount (20 GB), Storage for each RDS instance (General Purpose SSD (gp2)), Nodes (1), Instance type (db.t4g.micro), Utilization (On-Demand only) (100 %Utilized/Month), Deployment option (Single-AZ), Pricing strategy (OnDemand)

Name	Group	Region	Upfront cost	Monthly cost
AWS Secrets Manager	-	US East (N. Virginia)	0.00 USD	0.40 USD

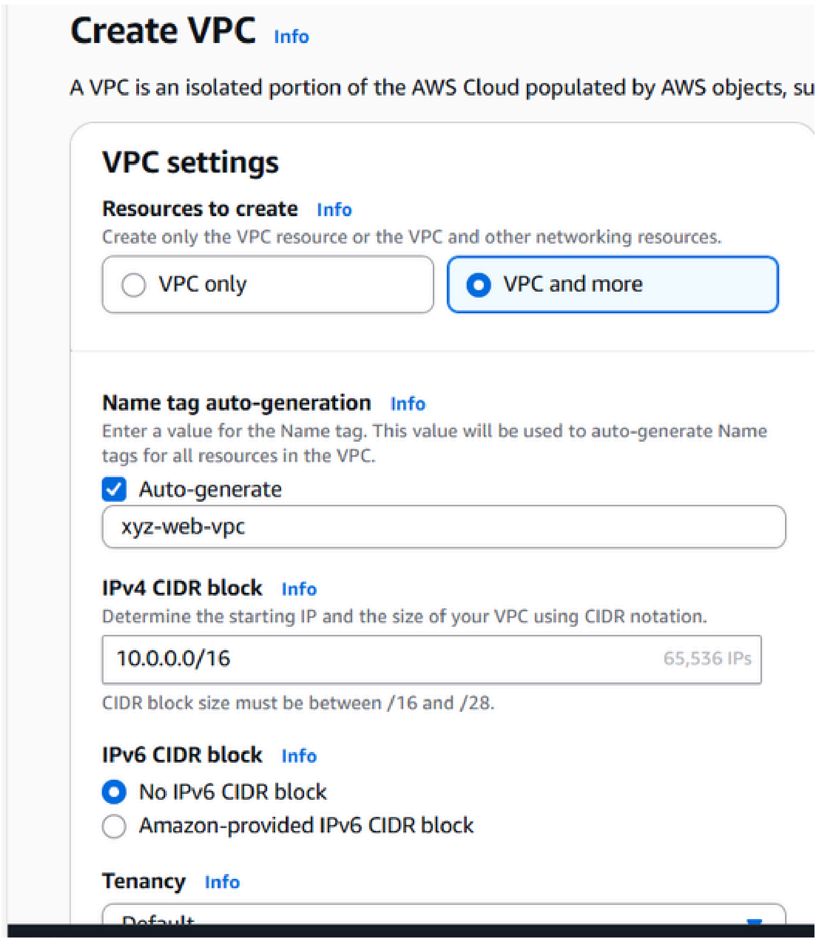
Status -

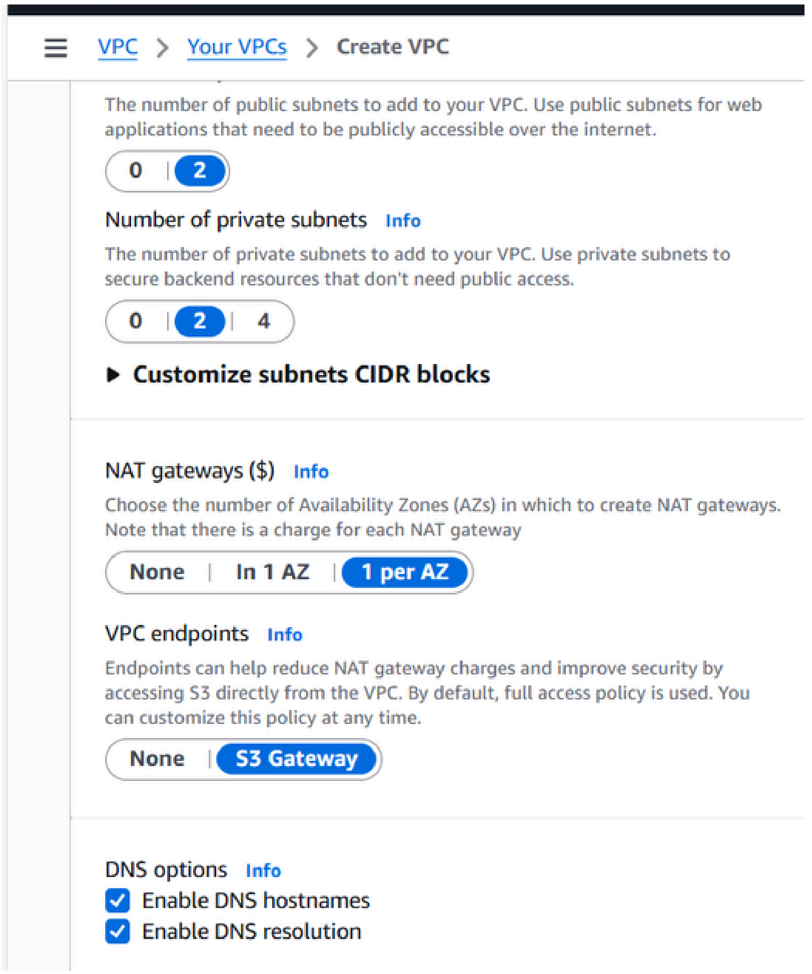
Description: -

Phase 2: Creating a basic functional web application

Task 1: Creating a virtual network

Create networking resources such as a virtual private cloud (VPC) and subnets.

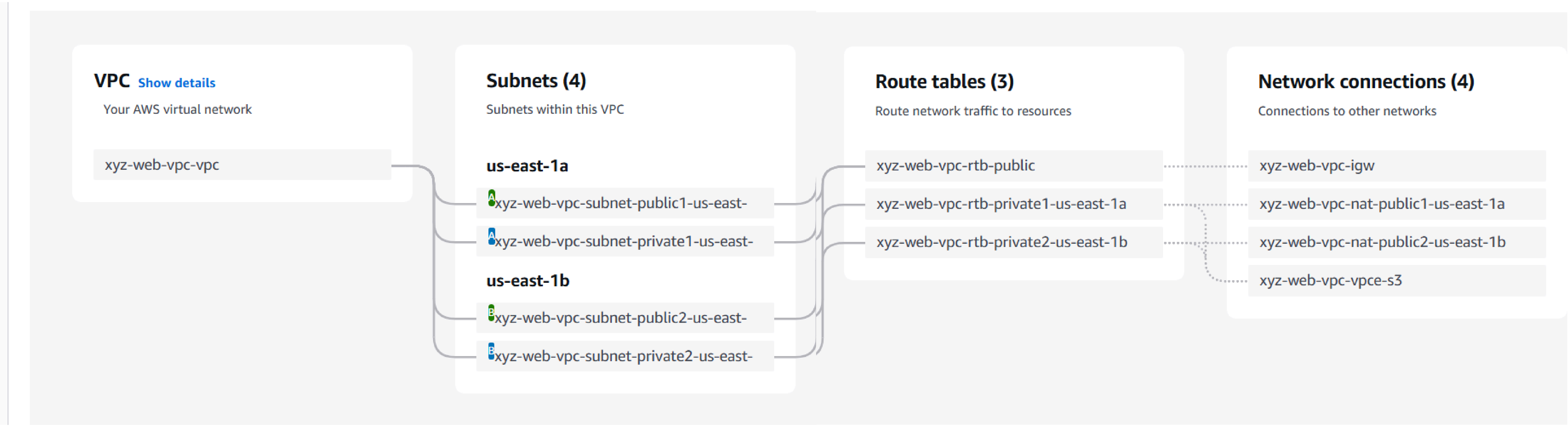




Phase 2: Creating a basic functional web application

Task A: Create VPC Components

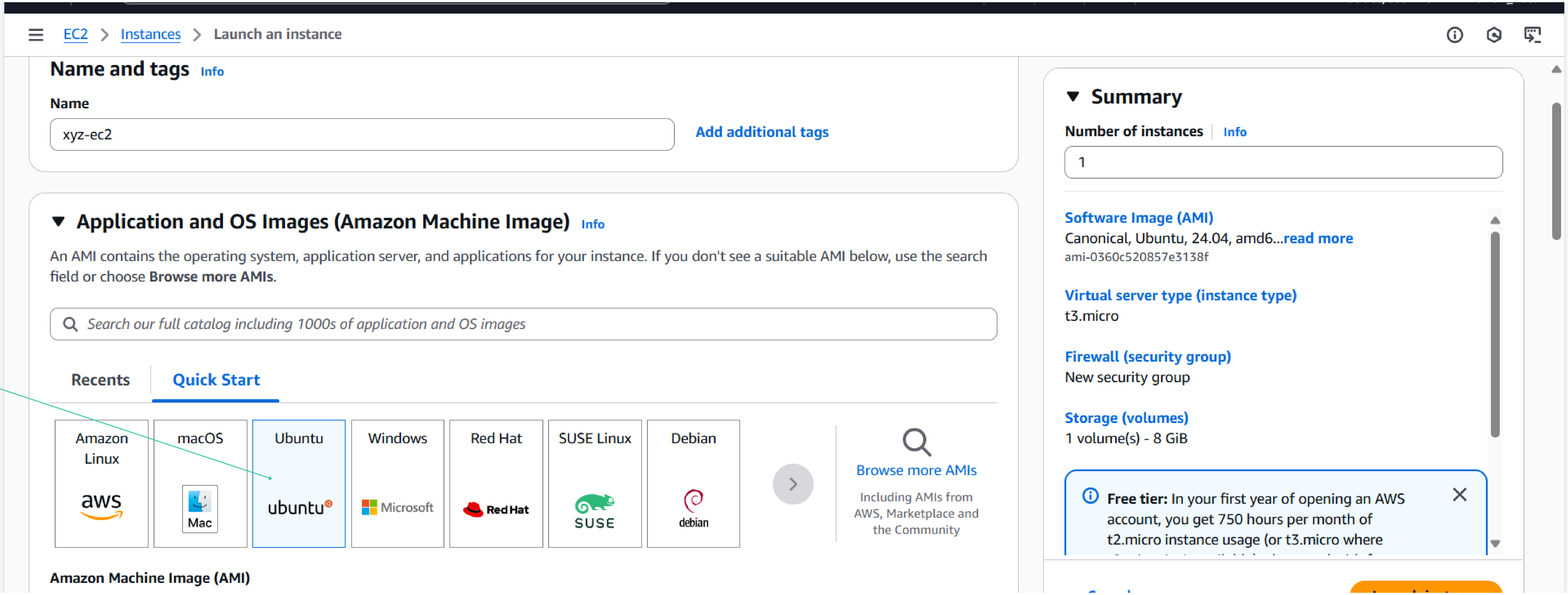
Now we will create subnets, an internet gateway, and route tables for our VPC.No need to change the VPC configuration



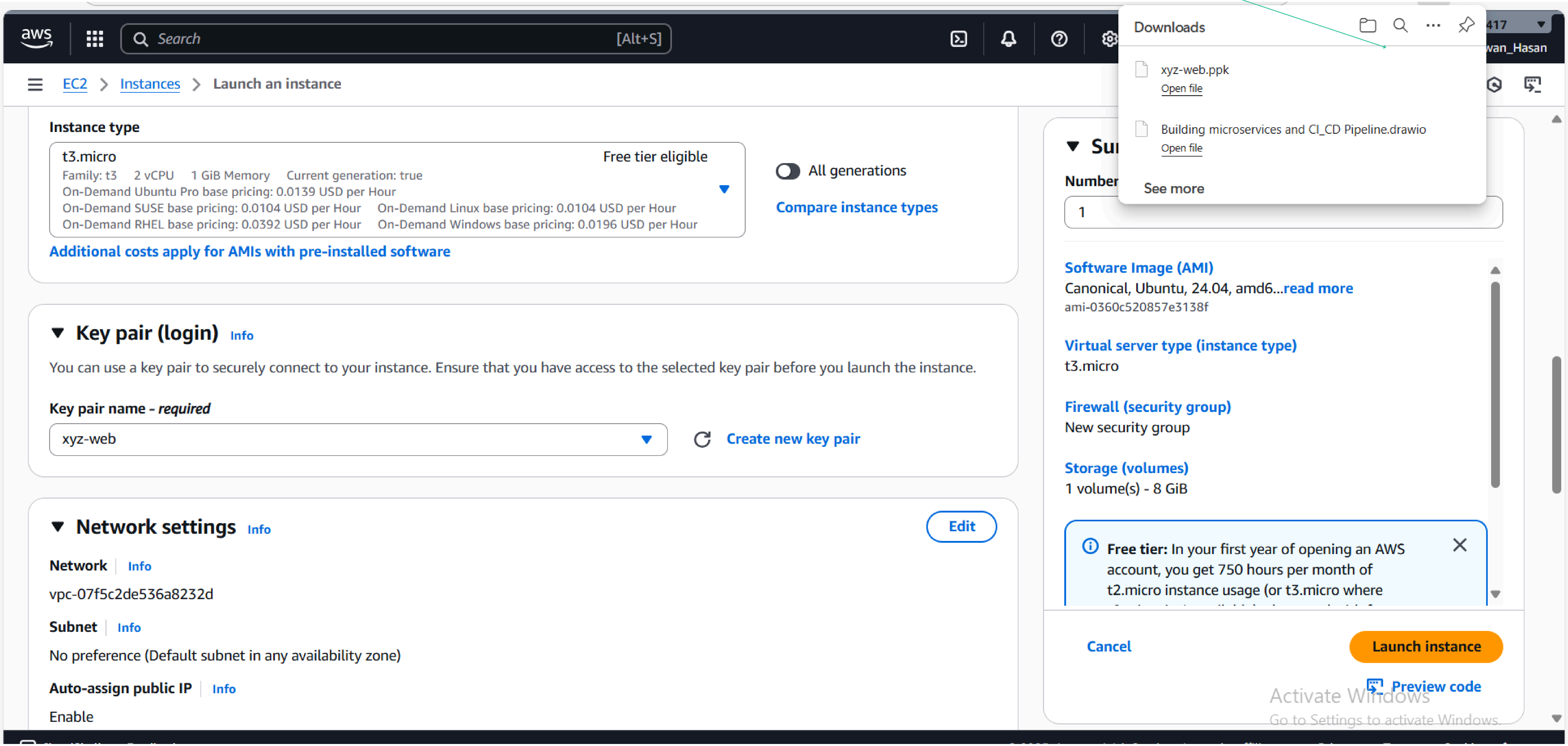
Phase 2: Creating a basic functional web application

Task 2: Creating a virtual machine

Create a Ubuntu EC2

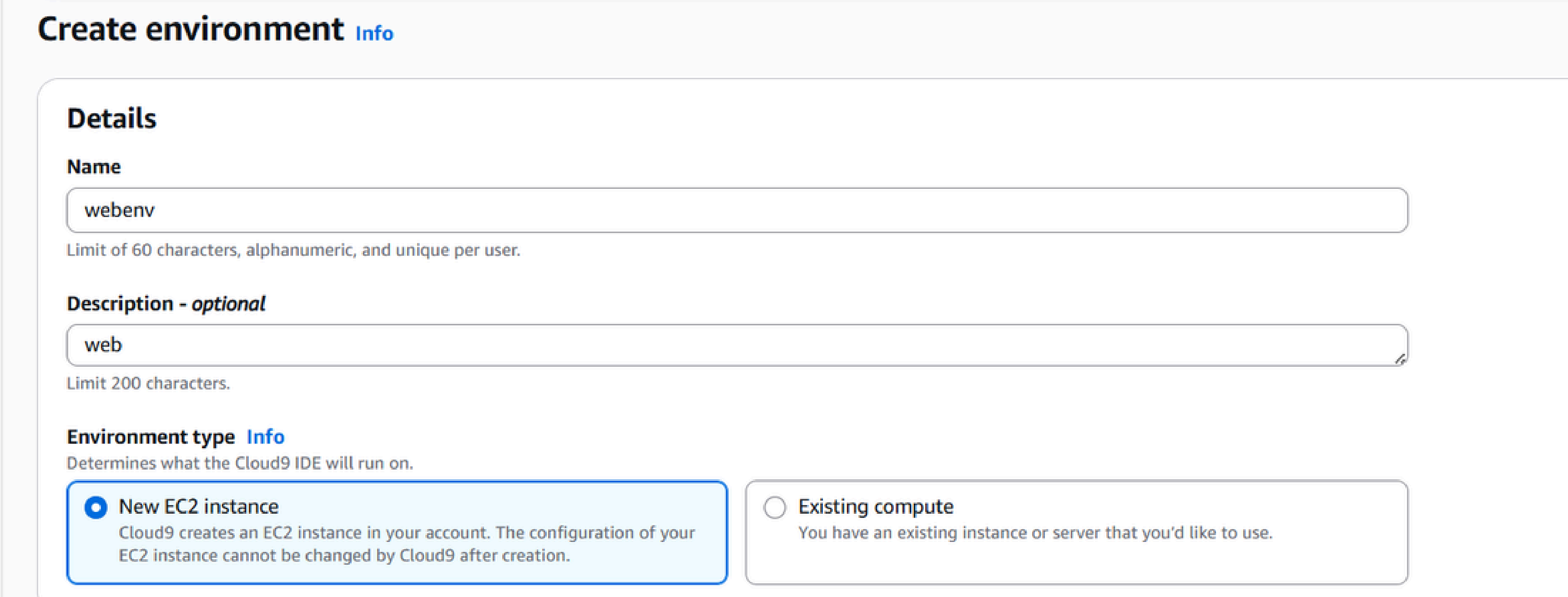


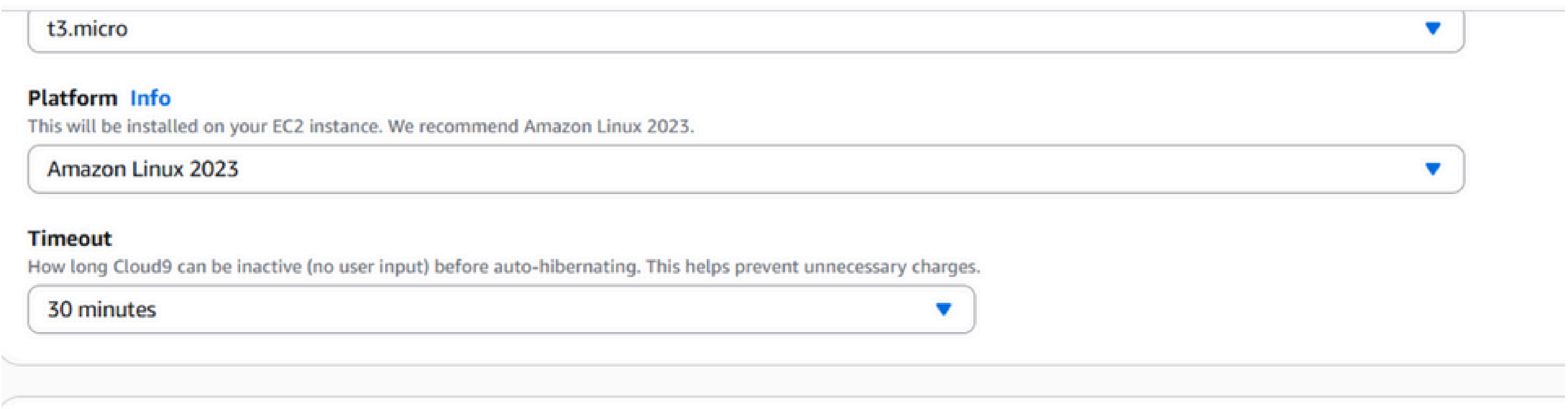
created a new key pair and downloaded it

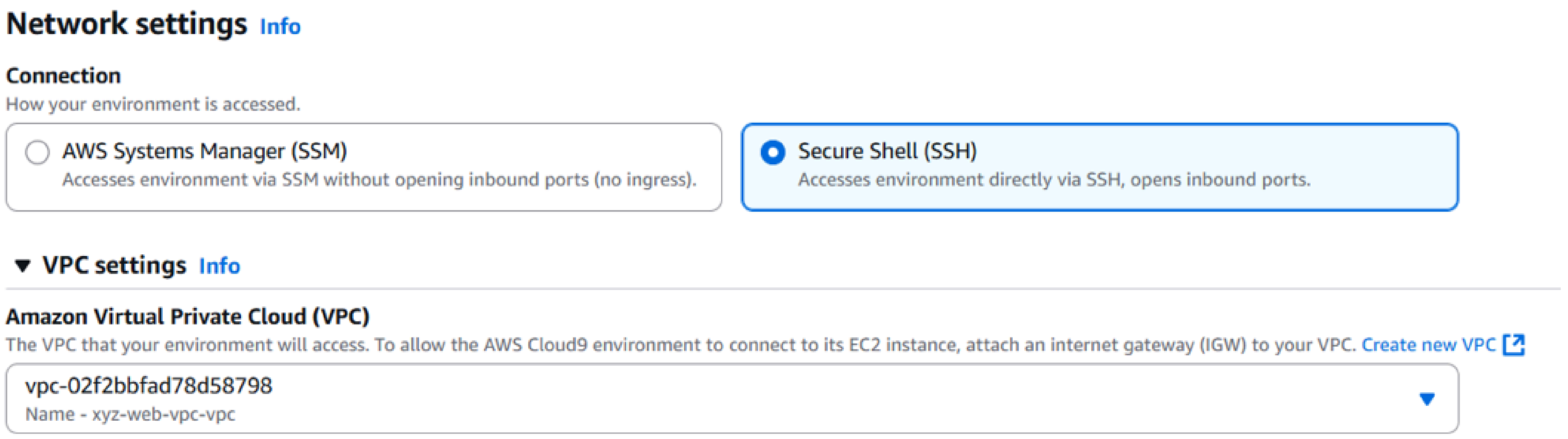


Configuring the development environment

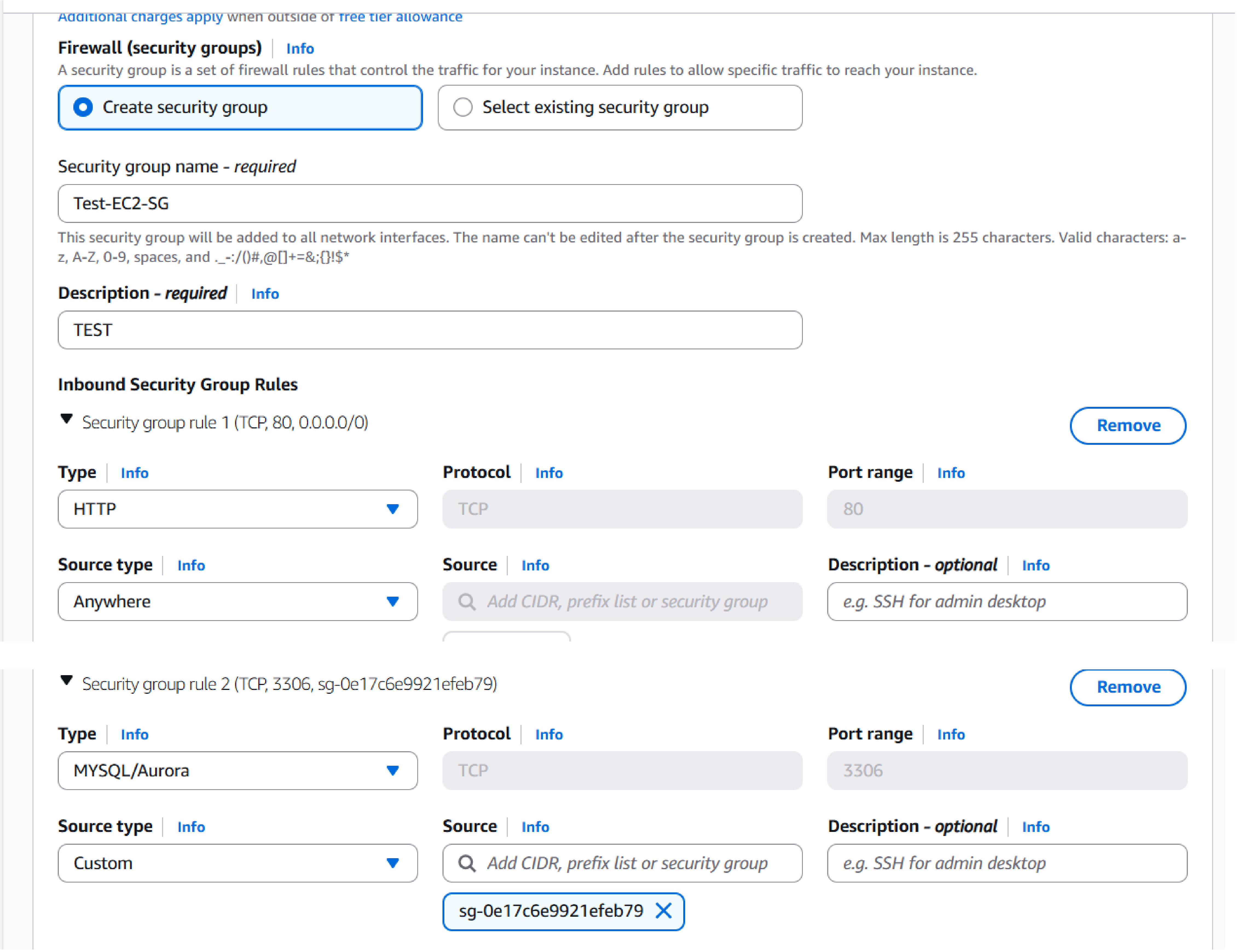
1. Go to AWS Cloud9 Services
2. Choose Create Environment
3. Enter the Name and Description
4. Select New EC2 instance for Cloud9 Environment
5. Instance Type is t3.micro
6. For the platform, choose Linux Server
7. In Network Settings:
   Choose SSH connection







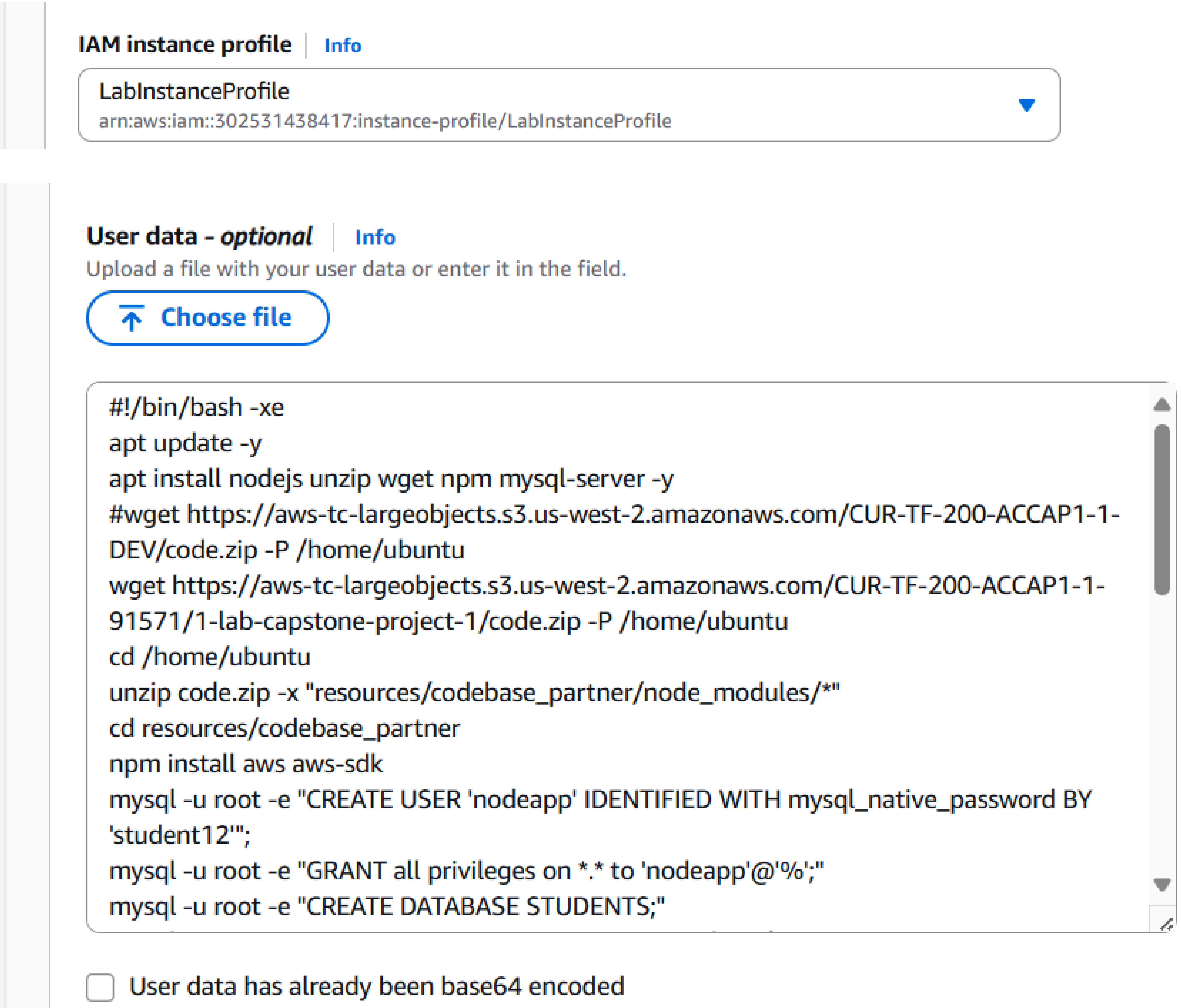
Create Security Group rules



Add IAM profile and copy the

Script UserdataScript-phase-2.sh

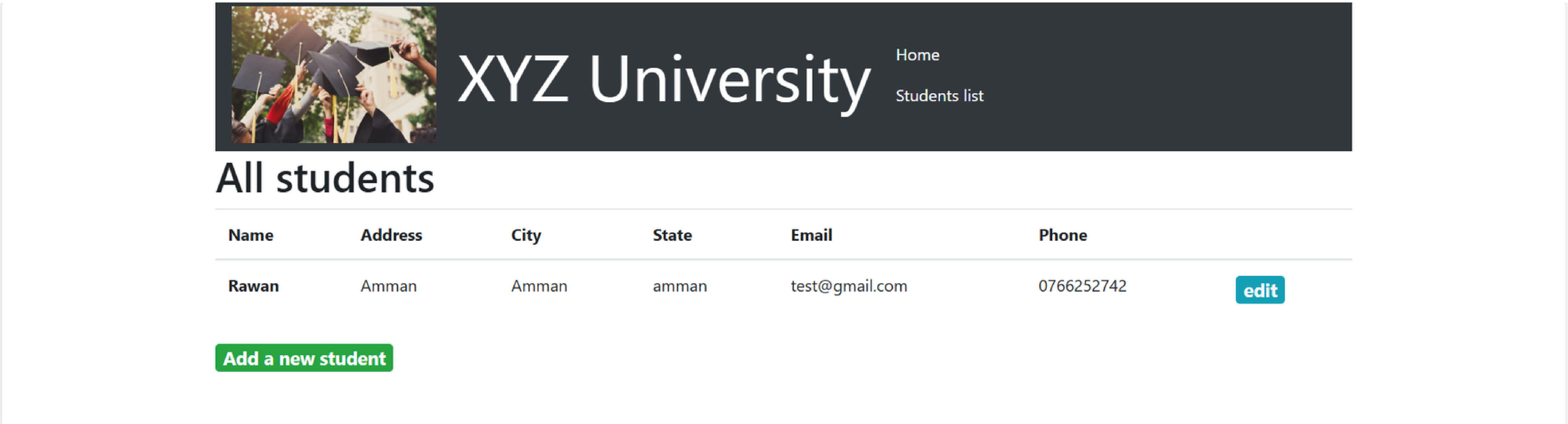
And paste it.

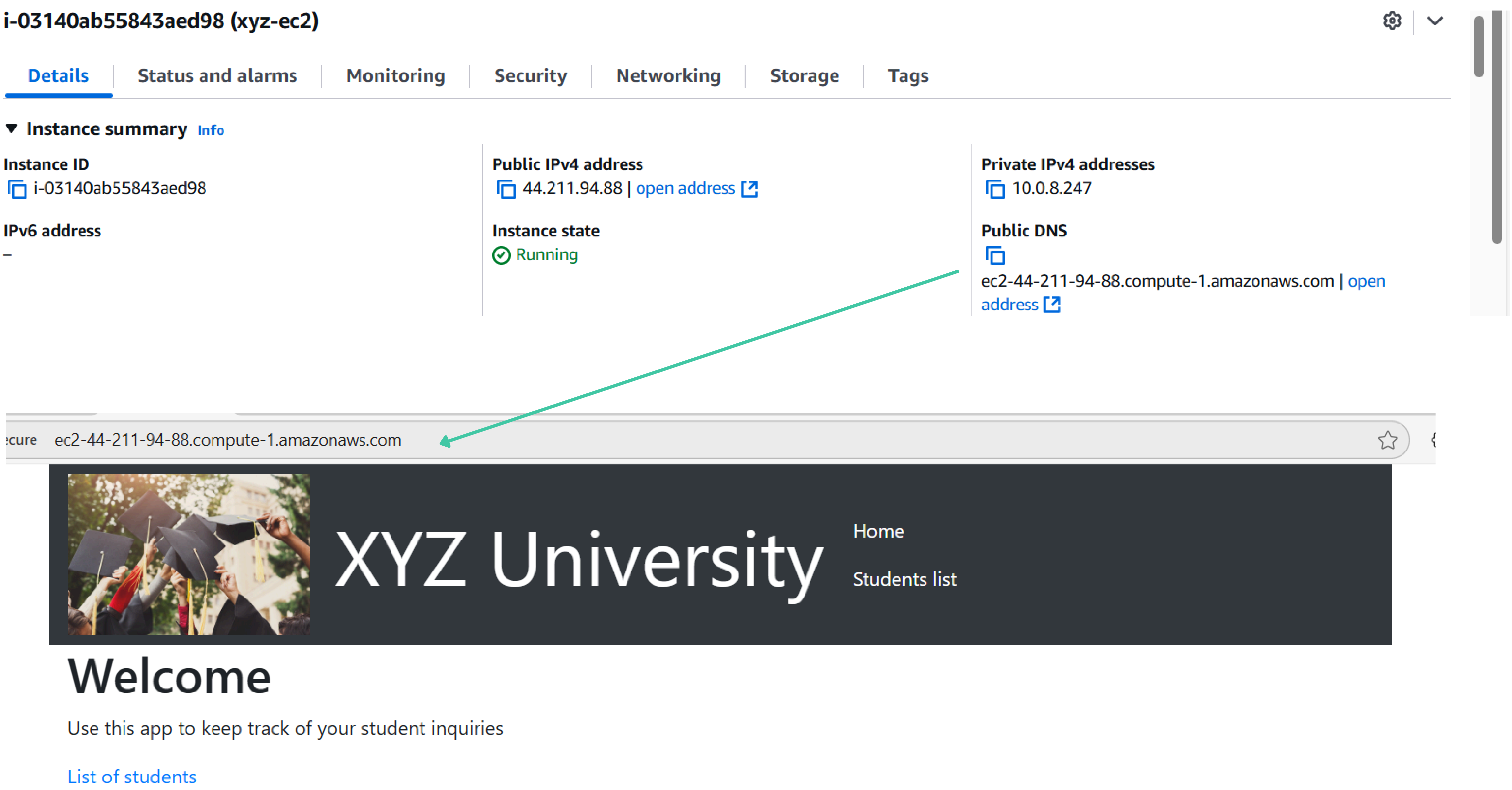


Phase 2: Creating a basic functional web application

- **Task 3: Testing the deployment**

Add a few dummy entries




i-03140ab55843aed98 (xyz-ec2)
Details
Status and alarms
Monitoring
Security
Networking
Storage
Tags
Instance summary Info
Instance ID
i-03140ab55843aed98
IPv6 address
–
Public IPv4 address
44.211.94.88 | open address
Instance state
Running
Private IPv4 addresses
10.0.8.247
Public DNS
ec2-44-211-94-88.compute-1.amazonaws.com | open address
ecure ec2-44-211-94-88.compute-1.amazonaws.com
XYZ University
Home
Students list
Welcome
Use this app to keep track of your student inquiries
List of students

Phase 3: Decoupling the application components

- **Task 1: Changing the VPC configuration**

No need to change

- **Task 2: Creating and configuring the Amazon RDS database**

Create an Amazon Relational Database Service (Amazon RDS) database that runs a MySQL engine. You can choose to create a provisioned instance or run it serverless.

RDS > Database

Create database Info

Choose a database creation method

- (selected) **Standard create**
  You set all of the configuration options, including ones for availability, security, backups, and maintenance.
- **Easy create**
  Use recommended best-practice configurations. Some configuration options can be changed after the database is created.

Engine options

Engine type Info

- Aurora (MySQL Compatible)
- Aurora (PostgreSQL Compatible)
- (selected) MySQL

Templates

Choose a sample template to meet your use case.

- **Production**
  Use defaults for high availability and fast, consistent performance.
- **Dev/Test**
  This instance is intended for development use outside of a production environment.
- (selected) **Free tier**
  Use RDS Free Tier to develop new applications, test existing applications, or gain hands-on experience with Amazon RDS. Info

Availability and durability

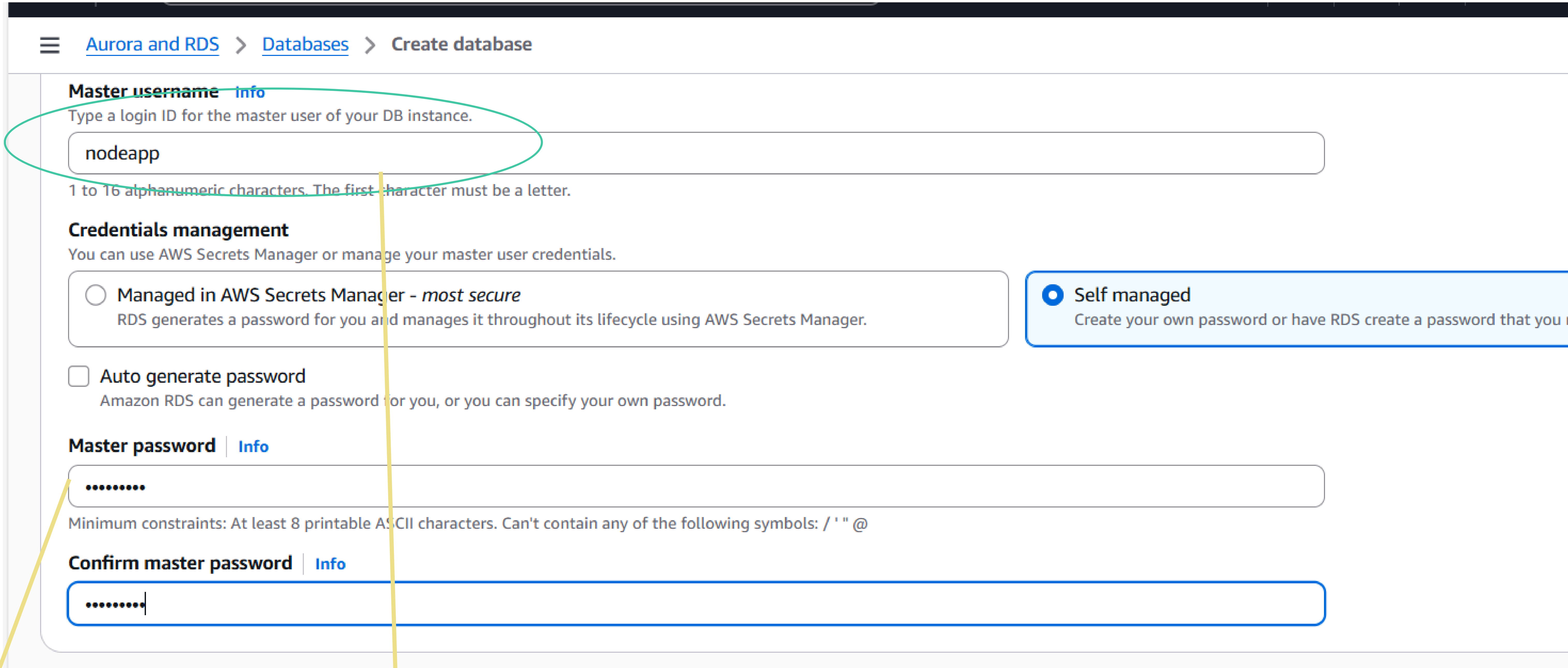
Deployment options Info

Choose the deployment option that provides the availability and durability needed for your use case. AWS is committed to a certain level of uptime depending on the deployment option you choose. Learn more in the Amazon RDS service level agreement (SLA).

- **Multi-AZ DB cluster deployment (3 instances)**
  Creates a primary DB instance with two readable standbys in separate Availability Zones. This setup provides:
  - 99.95% uptime
  - Redundancy across Availability Zones
  - Increased read capacity
  - Reduced write latency
- **Multi-AZ DB instance deployment (2 instances)**
  Creates a primary DB instance with a non-readable standby instance in a separate Availability Zone. This setup provides:
  - 99.95% uptime
  - Redundancy across Availability Zones
- (selected) **Single-AZ DB instance deployment (1 instance)**
  Creates a single DB instance without standby instances. This setup provides:
  - 99.5% uptime
  - No data redundancy

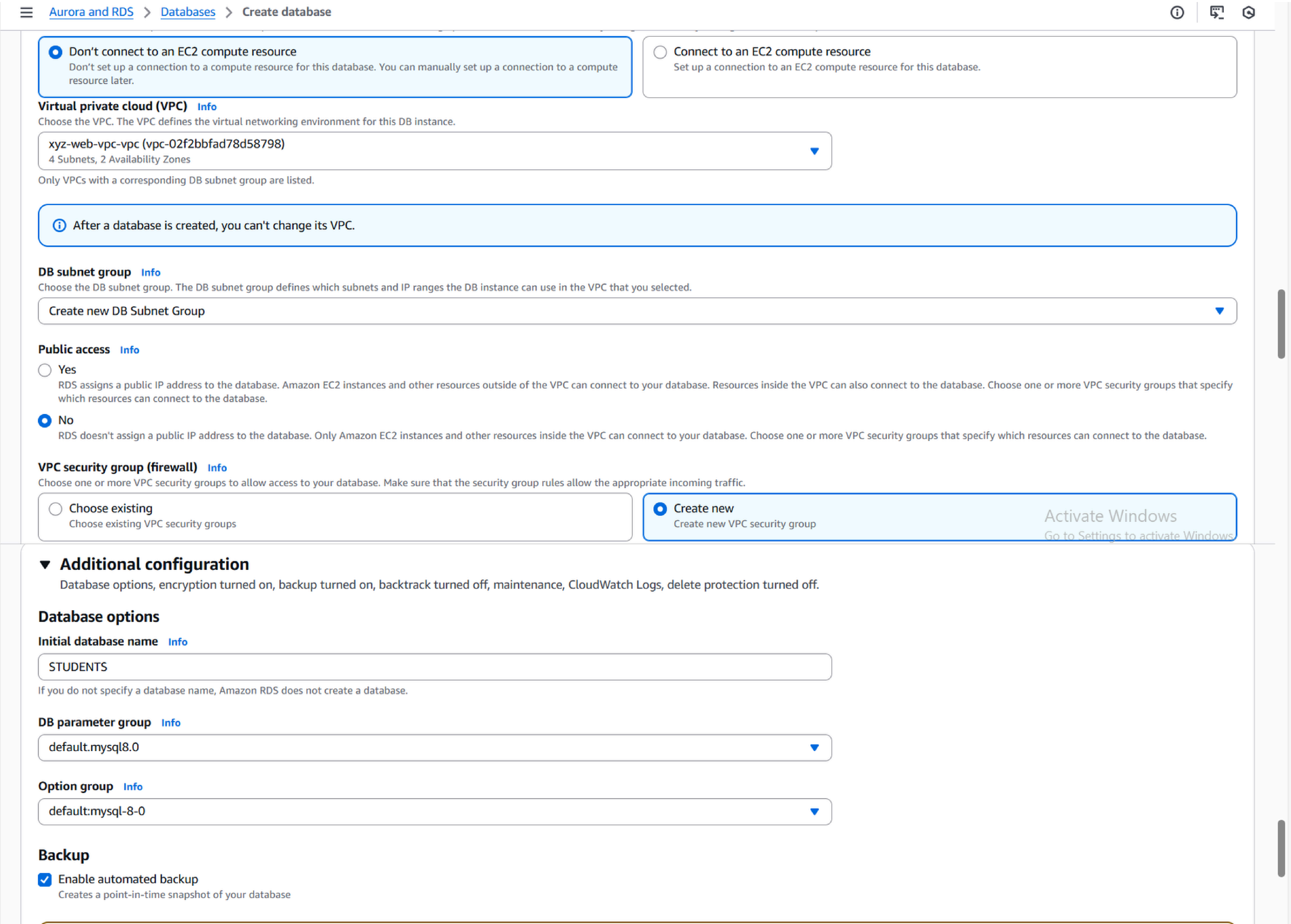
In Credential Settings

Enter your
Master Username
Master Password



mysql -u root -e "CREATE USER 'nodeapp' IDENTIFIED WITH mysql_native_password BY 'student12'";
mysql -u root -e "GRANT all privileges on *.* to 'nodeapp'@'%';"
mysql -u root -e "CREATE DATABASE STUDENTS;"

Create a subnet group
Create a security group



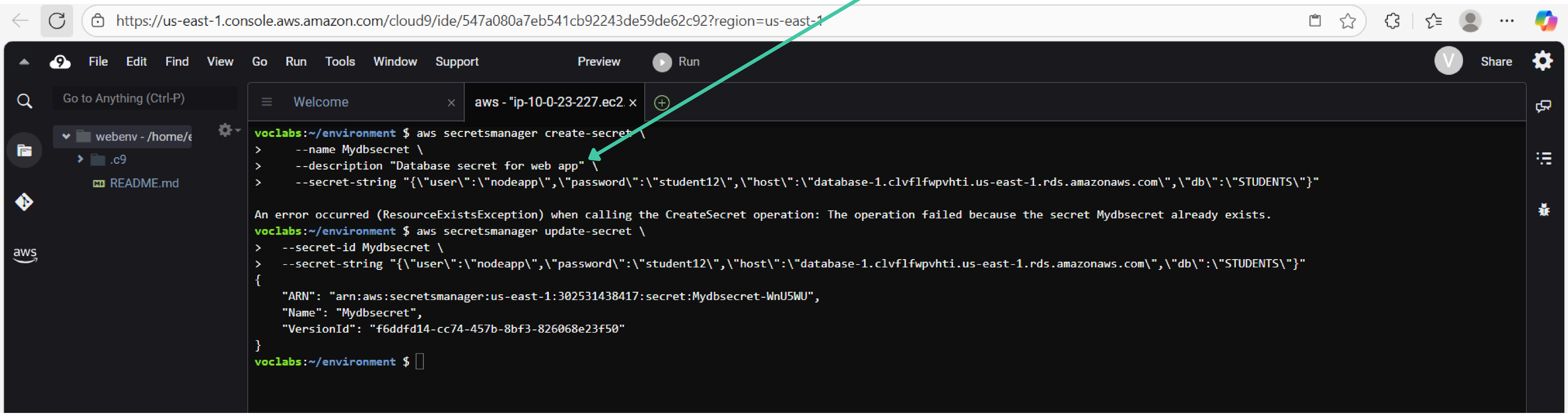
Phase 3: Decoupling the application components

- **Task 3:Configuring the development environment**

done in the previous step

- **Task 4: Provisioning Secrets Manager**

update to new



Cloud9

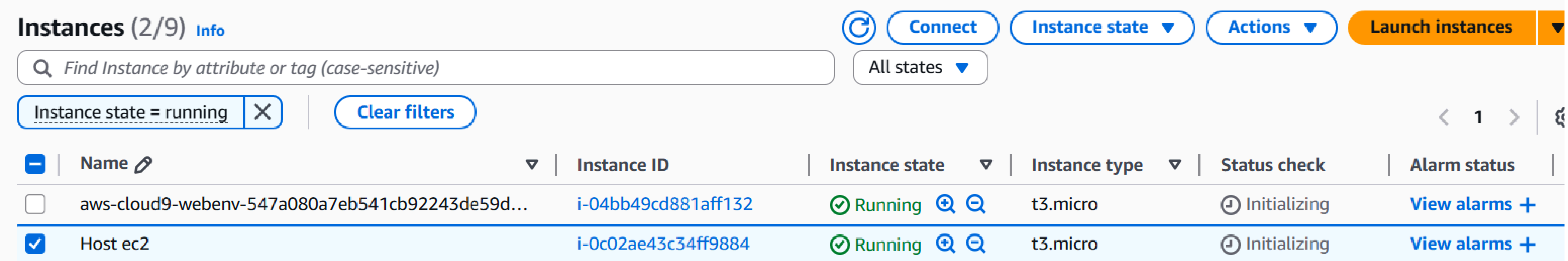
Script-1:

```
# Use following script to create secret in secret manager for accessing your database via the web application
# Replace the values for below placeholders with the actual values you have used to configure the service
#<RDS Endpoint>
#<password>
#<username>
#<dbname>
aws secretsmanager create-secret \
   --name Mydbsecret \
   --description "Database secret for web app" \
   --secret-string "{\"user\":\"<username>\",\"password\":\"<password>\",\"host\":\"<RDS Endpoint>\",\"db\":\"<dbname>\"}"
```

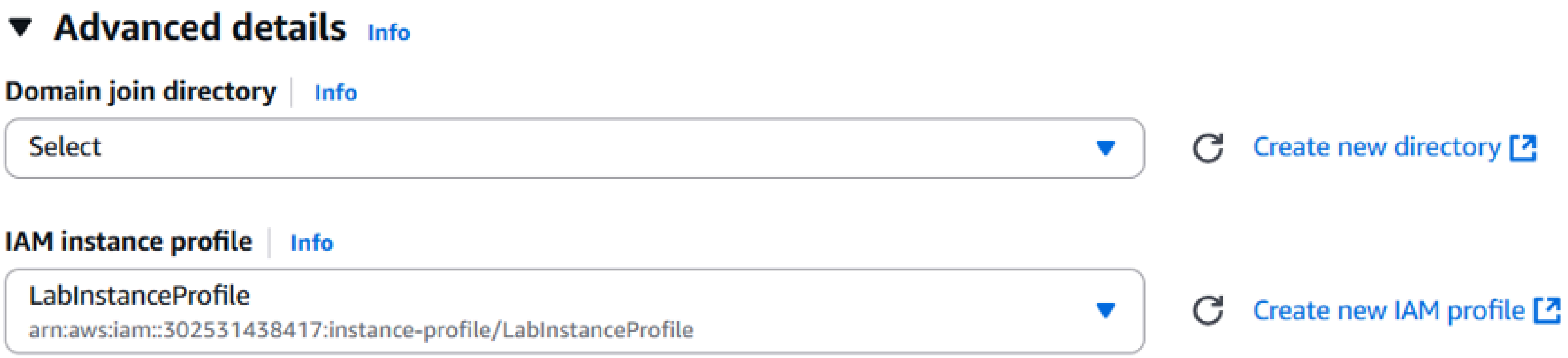
Phase 3: Decoupling the application components

- **Task 5: Provisioning a new instance for the web server**

Create a new virtual machine to host the web application.



Choose IAM instance profile in the Advanced details.



In the User data, give the UserdataScript-phase-3.sh script.

```
#!/bin/bash -xe
apt update -y
apt install nodejs unzip wget npm mysql-client -y
#wget https://aws-tc-largeobjects.s3.us-west-2.amazonaws.com/CUR-TF-200-ACCAP1-1-DEV/code.zip -P /home/ubuntu
wget https://aws-tc-largeobjects.s3.us-west-2.amazonaws.com/CUR-TF-200-ACCAP1-1-91571/1-lab-capstone-project-1/code.zip -P /home/ubuntu
cd /home/ubuntu
unzip code.zip -x "resources/codebase_partner/node_modules/*"
cd resources/codebase_partner
npm install aws aws-sdk
export APP_PORT=80
npm start &
```

Task 6: Migrating the database

- Migrate the data from the original database, which is on an EC2 instance, to the new Amazon RDS database.
- Use *Script-3* from the AWS Cloud9 Scripts file (cloud9-scripts.yml) to migrate the original data into the Amazon RDS database.

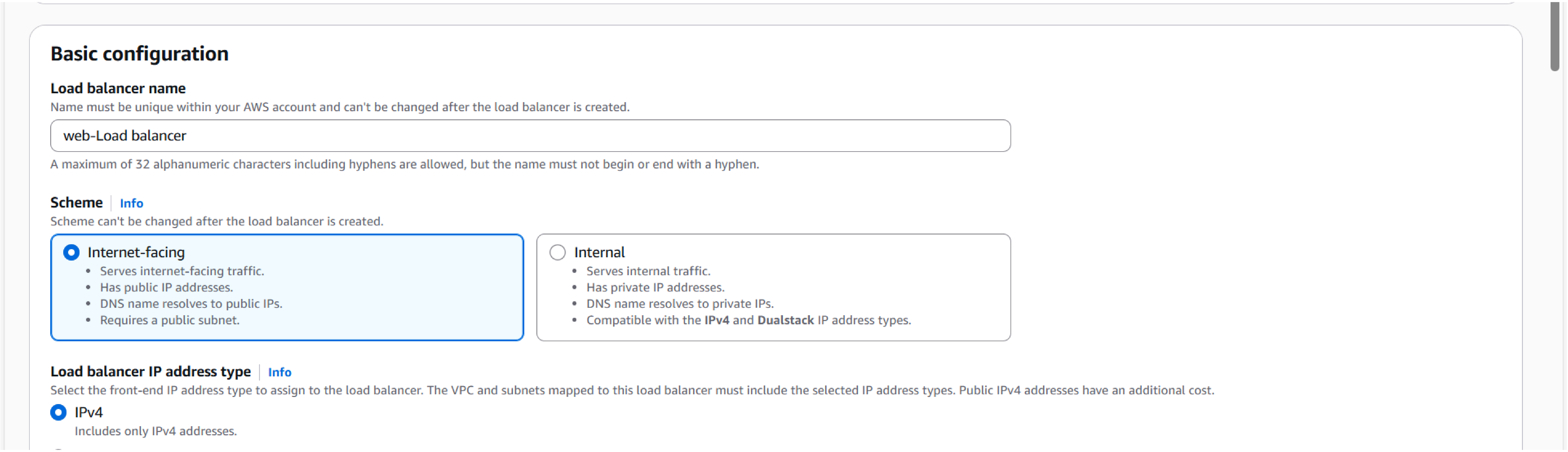
mysqldump -h 10.0.8.247 -u nodeapp -p --databases STUDENTS > data.sql

mysql -h database-1.clvflfwpvhti.us-east-1.rds.amazonaws.com -u nodeapp -p STUDENTS < data.sql

```
voclabs:~/environment $ aws secretsmanager update-secret \
>   --secret-id Mydbsecret \
>   --secret-string "{\"user\":\"nodeapp\",\"password\":\"student12\",\"host\":\"database-1.clvflfwpvhti.us-east-1.rds.amazonaws.com\",\"db\":\"STUDENTS\"}"
{
    "ARN": "arn:aws:secretsmanager:us-east-1:302531438417:secret:Mydbsecret-WnU5WU",
    "Name": "Mydbsecret",
    "VersionId": "f6ddfd14-cc74-457b-8bf3-826068e23f50"
}
voclabs:~/environment $ mysqldump -h 10.0.8.247 -u nodeapp -p --databases STUDENTS > data.sql
Enter password:
voclabs:~/environment $ mysql -h database-1.clvflfwpvhti.us-east-1.rds.amazonaws.com -u nodeapp -p  STUDENTS < data.sql
Enter password:
voclabs:~/environment $
```

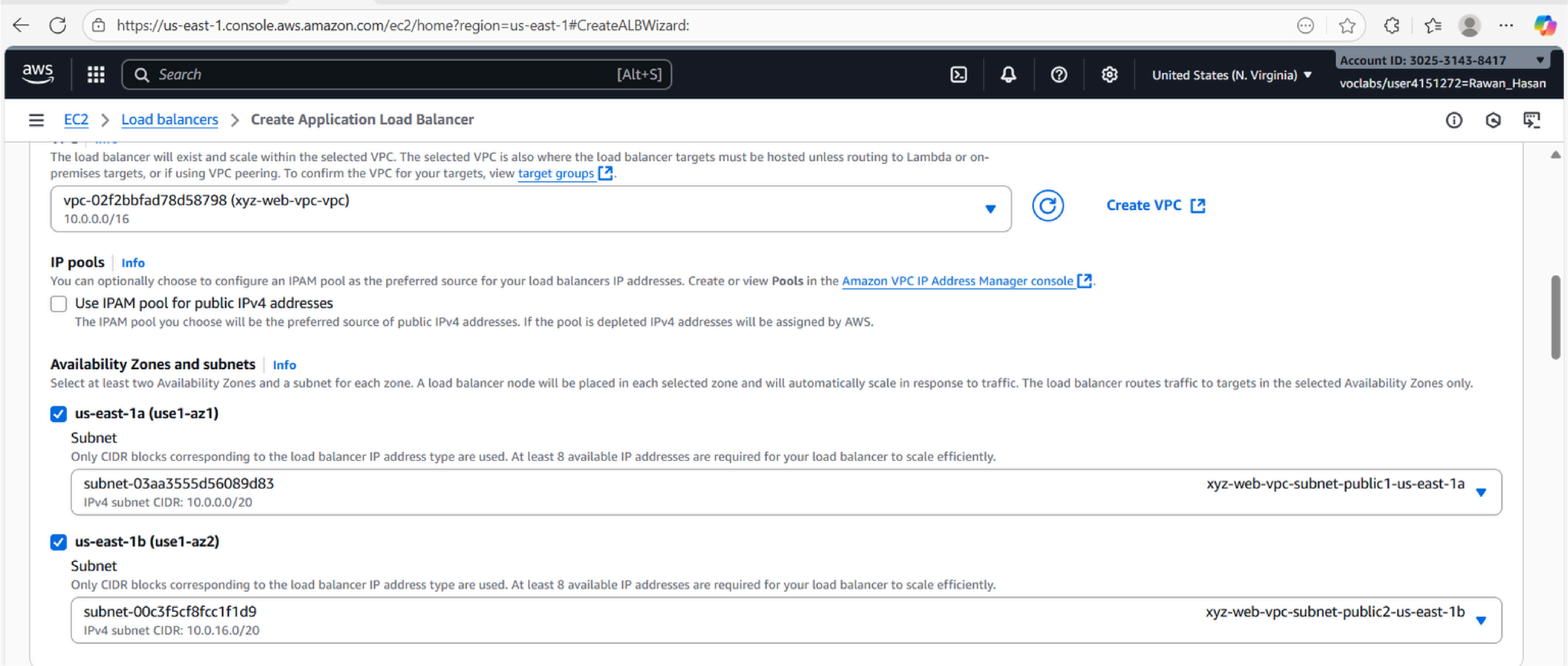
Phase 4: Implementing high availability and scalability

- **Task 1: Creating an Application Load Balancer**



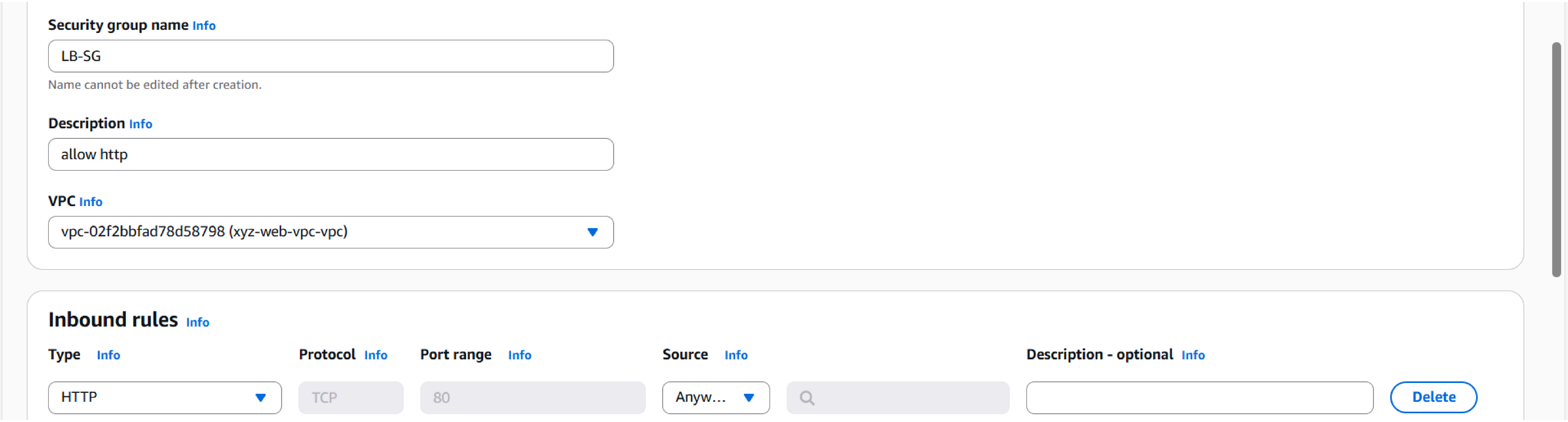
Creating a Load Balancer

- EC2 > Load Balancers > Create Application Load Balancer
- Assign a name to the Load balancer in Basic Configuration.
- In Network Mapping, choose your VPC.
- Under Mappings, choose first availability zone, then choose the Public Subnet.
- Choose the second availability zone, then choose the public Subnet.

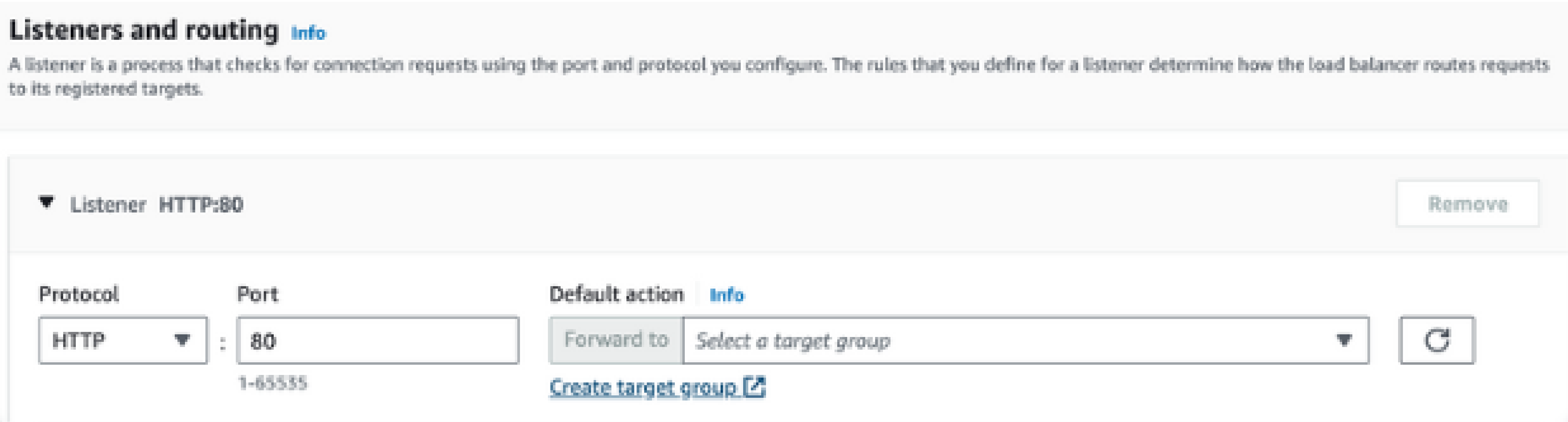


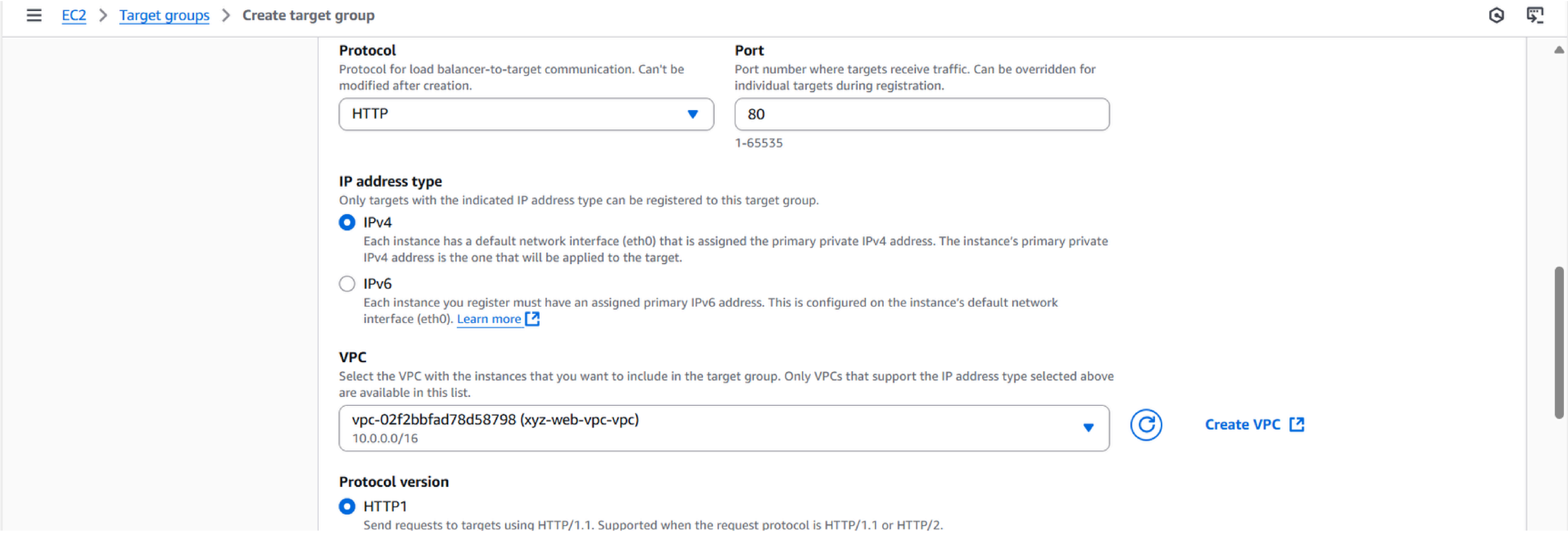
Creating a load Balancer

Create a new Security Group in Security Group



Create Target Group





https://us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#CreateTargetGroup:protocol=HTTP;vpc=vpc-02f2bbfad78d58798

aws Search [Alt+S]

United States (N. Virginia) ▼

Account ID: 3025-3143-8417 ▼

voclabs/user4151272=Rawan_Hasan

Cloud9

EC2 > Target groups > Create target group

0 selected

Ports for the selected instances

Ports for routing traffic to the selected instances.

80

1-65535 (separate multiple ports with commas)

Include as pending below

1 selection is now pending below. Include more or register targets when ready.

Review targets

Targets (1)

Remove all pending

Filter targets

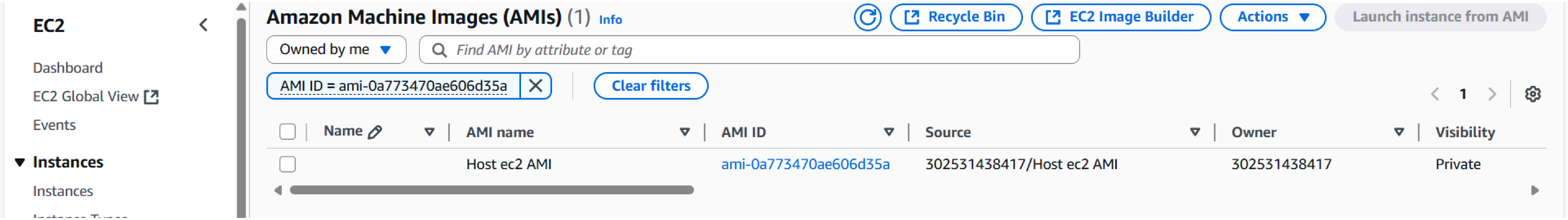
Show only pending

1

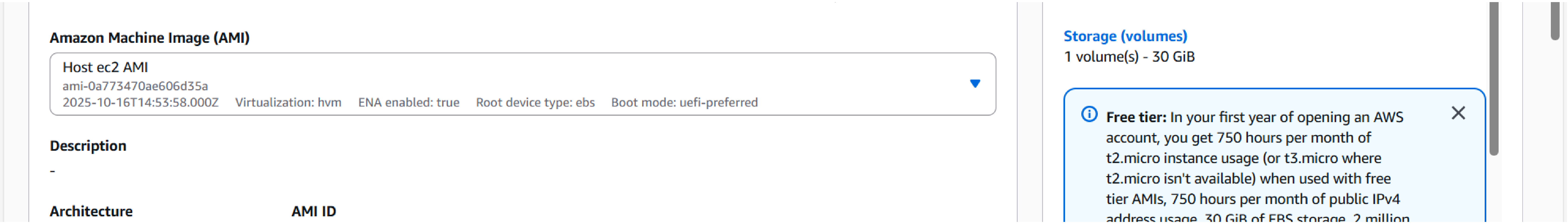
Instance ID	Name	Port	State	Security groups	Zone	Private IPv4 address	Subnet ID
i-0c02ae43c34ff9884	Host ec2	80	Running	launch web	us-east-1b	10.0.28.240	subnet-00c3f5cf8fcc1f1d9

Creating AMI

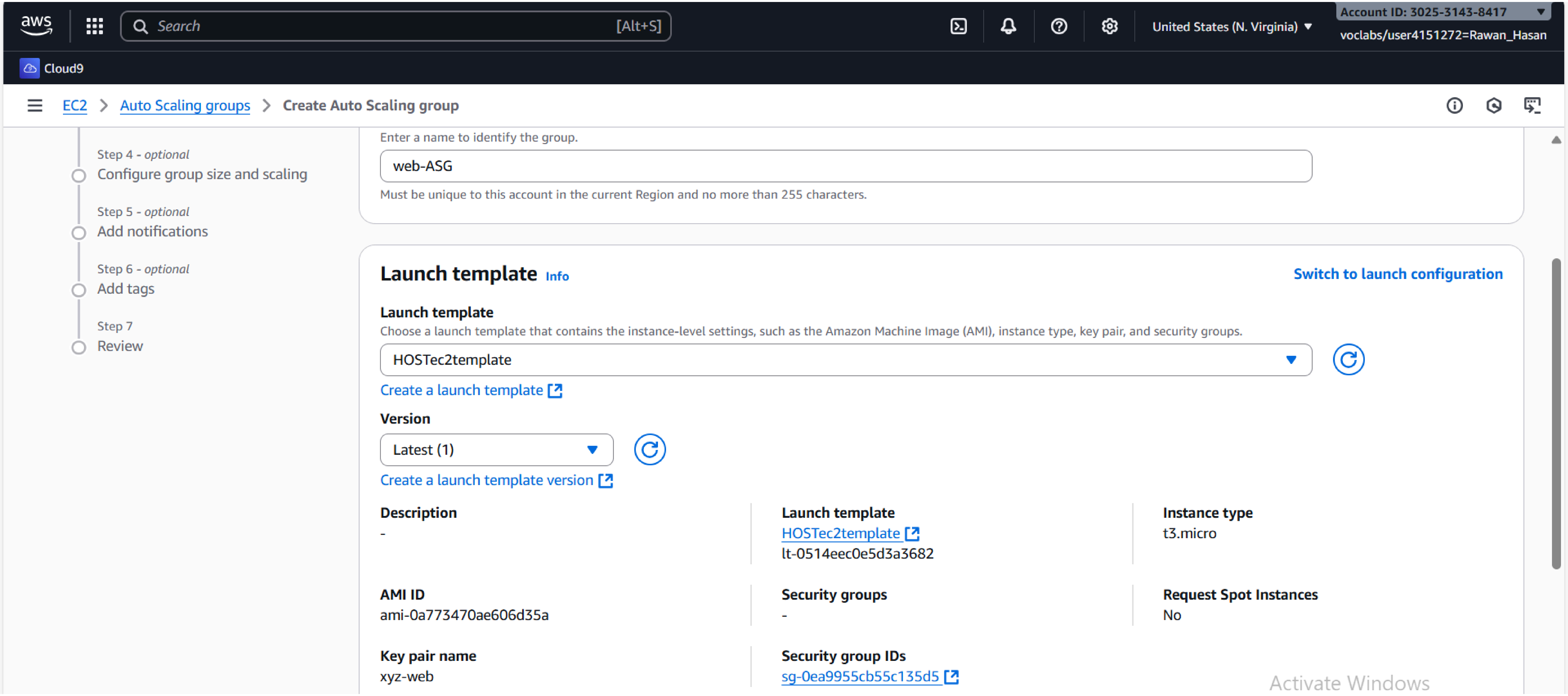
EC2 instance > Actions > Image and template > Create image
Image name: Host ec2 AMI

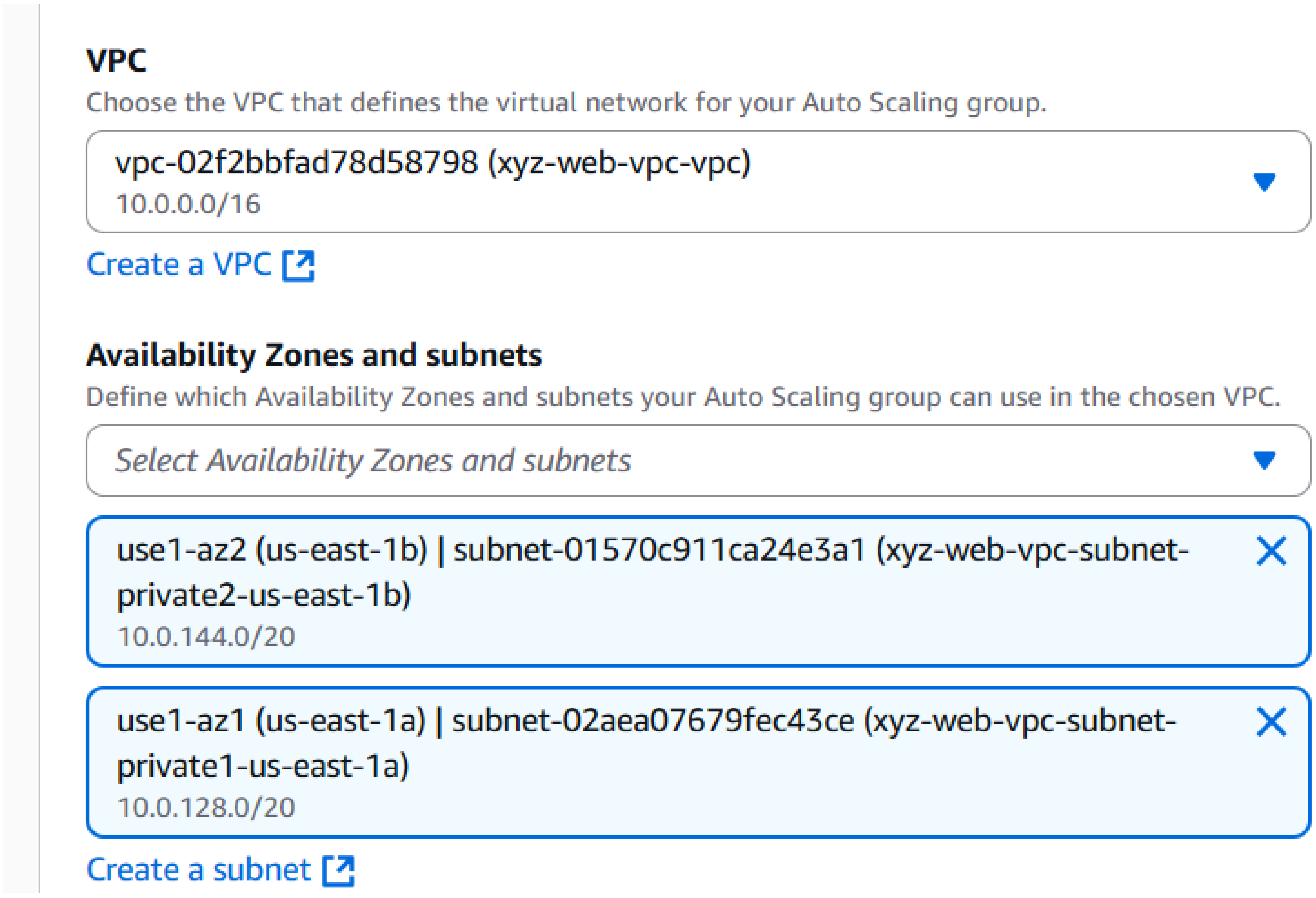


Create a Launch Template



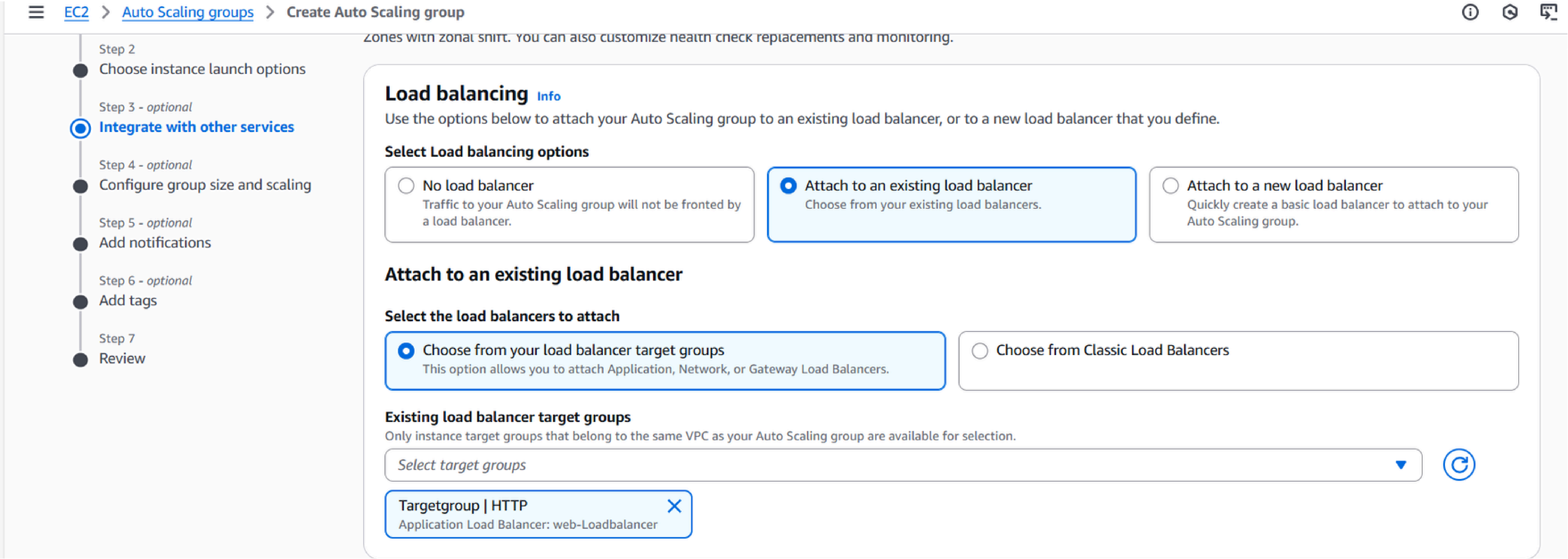
Creating Auto Scaling Group

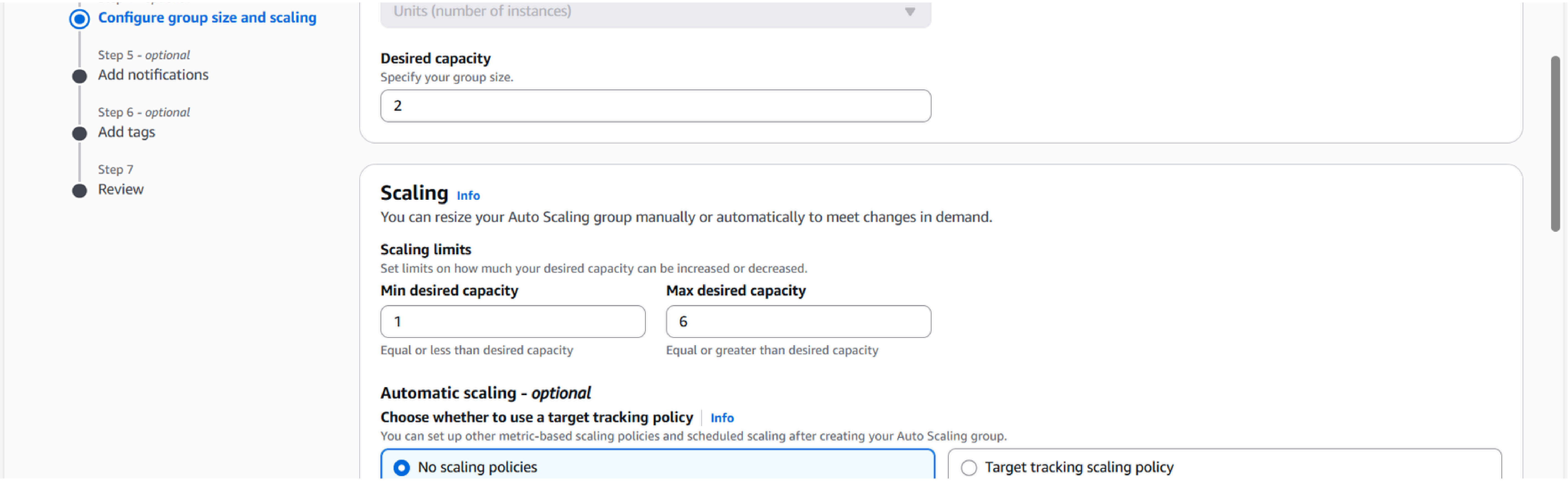




Choose a VPC and for Availability zones and Subnets

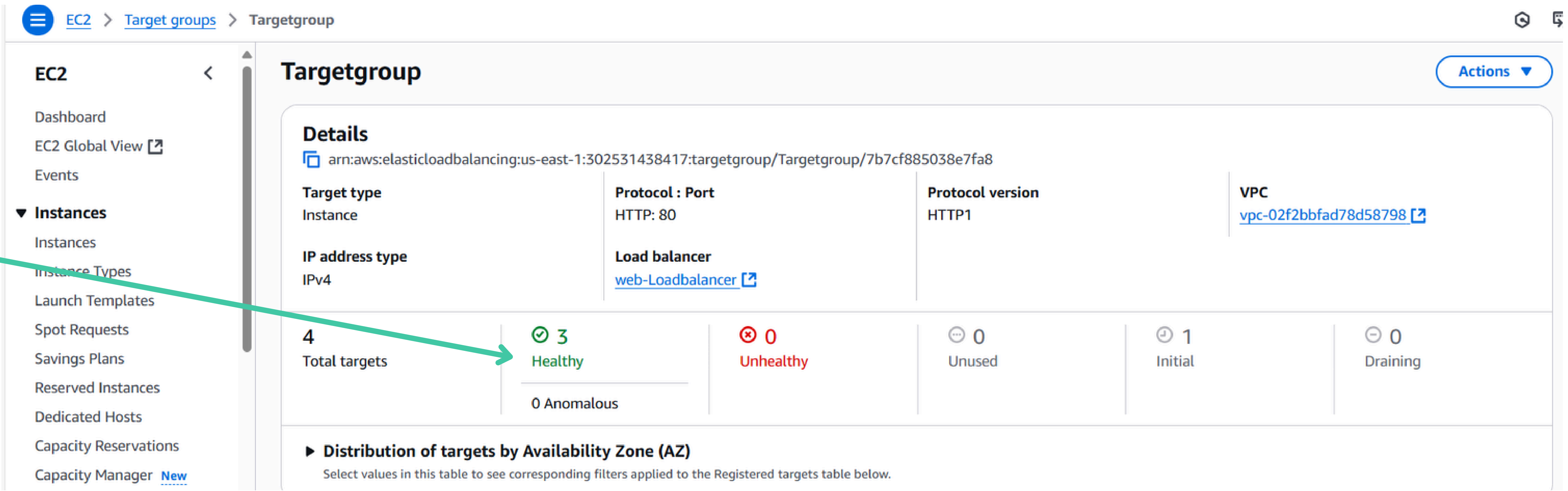
Choose *Private Subnet 1* and then choose *Private Subnet 2*. This will launch EC2 instances in private subnets across both Availability Zones.


EC2 > Auto Scaling groups > Create Auto Scaling group
Step 2
Choose instance launch options
Step 3 - optional
Integrate with other services
Step 4 - optional
Configure group size and scaling
Step 5 - optional
Add notifications
Step 6 - optional
Add tags
Step 7
Review
Zones with zonal shift. You can also customize health check replacements and monitoring.
Load balancing Info
Use the options below to attach your Auto Scaling group to an existing load balancer, or to a new load balancer that you define.
Select Load balancing options
No load balancer
Traffic to your Auto Scaling group will not be fronted by a load balancer.
Attach to an existing load balancer
Choose from your existing load balancers.
Attach to a new load balancer
Quickly create a basic load balancer to attach to your Auto Scaling group.
Attach to an existing load balancer
Select the load balancers to attach
Choose from your load balancer target groups
This option allows you to attach Application, Network, or Gateway Load Balancers.
Choose from Classic Load Balancers
Existing load balancer target groups
Only instance target groups that belong to the same VPC as your Auto Scaling group are available for selection.
Select target groups
Targetgroup | HTTP
Application Load Balancer: web-Loadbalancer


Configure group size and scaling
Step 5 - optional
Add notifications
Step 6 - optional
Add tags
Step 7
Review
Units (number of instances)
Desired capacity
Specify your group size.
2
Scaling Info
You can resize your Auto Scaling group manually or automatically to meet changes in demand.
Scaling limits
Set limits on how much your desired capacity can be increased or decreased.
Min desired capacity
1
Equal or less than desired capacity
Max desired capacity
6
Equal or greater than desired capacity
Automatic scaling - optional
Choose whether to use a target tracking policy Info
You can set up other metric-based scaling policies and scheduled scaling after creating your Auto Scaling group.
No scaling policies
Target tracking scaling policy

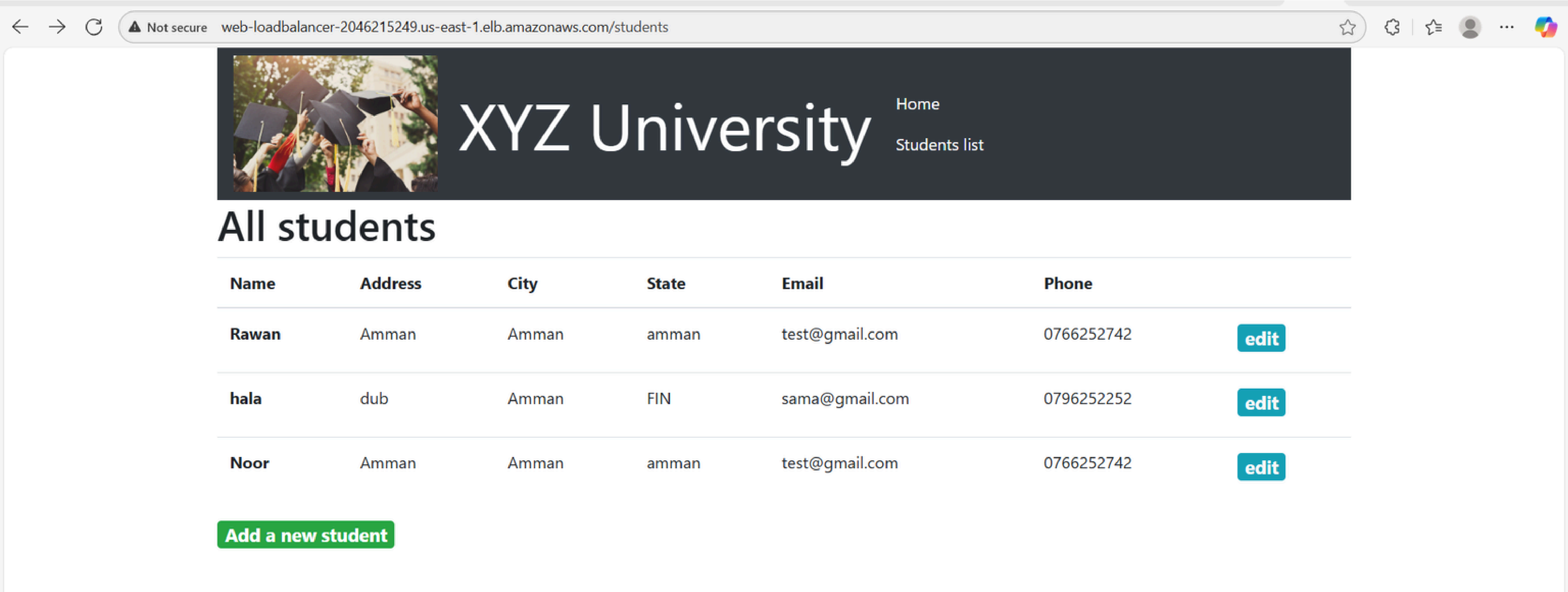
Testing the Application

The **Health status** column shows the results of the load balancer health check that is performed against the instances.

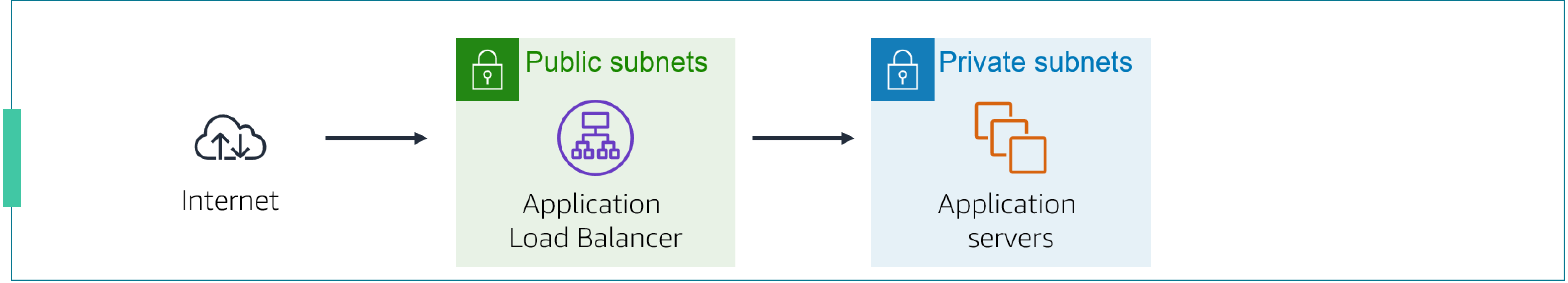


Testing the Application

In the load balancer details tab copy the DNS name.



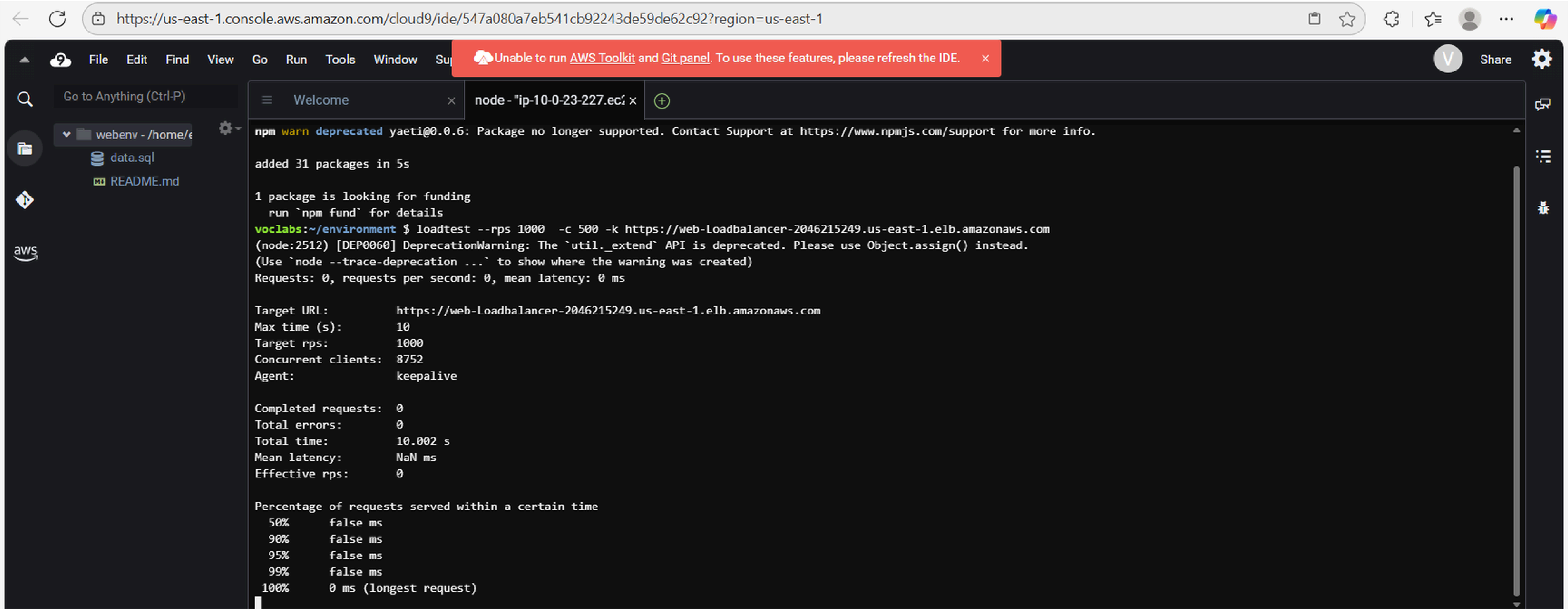
You should notice that the instance ID and Availability Zone sometimes toggles between the two instances.



- When this web application displays, the flow of data over the network is shown above.
- You sent the request to the *load balancer*, which resides in the *public subnets* that are connected to the internet.
- The load balancer chose one of the *EC2 instances* that reside in the *private subnets* and forwarded the request to it.
- The EC2 instance then returned the webpage content to the load balancer, which returned it to your web browser.

Testing the application in Cloud9

On running the Script-2 for load testing using load balancer DNS



loadtest --rps 1000 -c 500 -k https://web-Loadbalancer-2046215249.us-east-1.elb.amazonaws.com

Thank you